CocoConnect

Technology Stack Documentation

Client: Coconut Growers Association of Sri Lanka · Group 38

Prepared by: S.N. Vidanage (2023/IS/108)

1. Purpose

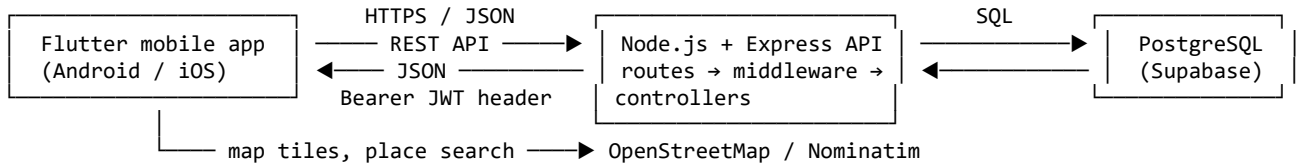
This document describes the technology stack chosen for CocoConnect: what each technology is, what it is used for, how the parts connect, and how the project is organised.

2. Stack Overview

Layer	Technology	Version	Role
Mobile app	Flutter (Dart)	Flutter 3.44, Dart 3.12	All screens, for Android and iOS from one codebase
Backend	Node.js + Express.js	Node.js 24, Express 5	REST API and business logic
Database	PostgreSQL, hosted on Supabase	PostgreSQL (Supabase)	Stores users, listings, requests, chats, orders
Authentication	JSON Web Tokens + bcrypt	jsonwebtoken 9, bcryptjs 3	Login tokens, role-based access, password hashing
Maps and location	OpenStreetMap (flutter_map), geolocator, Nominatim	flutter_map 8	Map location picker, GPS, place names, distance
API documentation	Swagger (OpenAPI)	swagger-ui-express 5	Interactive API docs at /api-docs
Testing	Node test runner + pg-mem	Node 24	Automated backend tests on an in-memory database
Version control	Git + GitHub	–	Source code, branches per member

3. Architecture

CocoConnect follows a client–server architecture with three tiers:



- **Presentation tier:** the Flutter app shows the screens and sends requests.
- **Application tier:** the Express API checks the login and role, applies the business rules and talks to the database.
- **Data tier:** PostgreSQL stores all data, with foreign keys between related tables and transactions for deals.

4. Frontend – Flutter

Flutter builds Android and iOS apps from a single Dart codebase and draws its own widgets, so the app looks the same on every phone.

Package	Used for
http	Calling the backend REST API
shared_preferences	Keeping the login token on the phone so users stay logged in
flutter_map + latlong2	Showing OpenStreetMap maps and picking a location
geolocator	Reading the phone's GPS position
flutter_lints	Code quality checks

Project structure

```
lib/
  main.dart        app entry point
  screens/         one file per screen (buyer/, orders/, chat/ ...)
  services/        API calls per area (auth, buyer, owner, chat ...)
  widgets/         reusable UI parts (cards, bottom bar, filters)
```

5. Backend – Node.js and Express.js

Node.js runs JavaScript on the server and handles many requests at once without blocking. Express.js organises the API into routes, middleware and controllers.

Package	Used for
express	Web server and REST routes
pg	Connecting to and querying PostgreSQL
bcryptjs	Hashing passwords before they are saved
jsonwebtoken	Creating and checking login tokens
cors	Allowing the app and the web portal to call the API
dotenv	Reading the database address and secrets from a private .env file
swagger-jsdoc, swagger-ui-express	API documentation
nodemon (development)	Restarting the server automatically when code changes
pg-mem (testing)	An in-memory PostgreSQL for tests

Project structure

```
backend/
  server.js        starts Express and mounts the routes
  config/          database connection, Swagger setup
  routes/          URL definitions per area (auth, buyer, owner, chat)
  middleware/      login and role checks
  controllers/     business logic and SQL queries
  utils/           shared helpers (distance and date calculations)
  sql/             database changes, numbered in order
  test/            automated tests
```

6. Database – PostgreSQL on Supabase

- PostgreSQL is a relational database. CocoConnect's data is closely related: a deal links a buyer, a listing, an estate and an order, so foreign keys and joins fit well.
- Transactions make multi-step actions safe: when an offer is accepted, the order is created and the other offers are closed together, or not at all.
- Supabase hosts the database in the cloud with a free tier, so the whole team works against the same data.
- Database changes are kept as numbered SQL files, so every copy of the database can be brought to the same structure.

7. Security

Concern	How it is handled
Passwords	Hashed with bcrypt before saving; the plain password is never stored
Login	The server issues a signed JWT that the app sends with every request
Access control	Middleware checks the token and the user's role before any protected route runs
Secrets	Database address and token secret are kept in a .env file that is never committed to GitHub
Location privacy	Exact coordinates stay on the server; other users only see the distance

8. Communication

- The app and the backend communicate through a REST API using JSON.
- Each request carries the login token in the header: Authorization: Bearer <token>.
- Errors come back with a clear message that the app shows to the user.
- Every route is documented in Swagger at /api-docs, where it can also be tried out.

9. Development Tools

Tool	Used for
Visual Studio Code	Writing code for the app and the backend
Android Studio + Android emulator	Android SDK and running the app on a virtual phone
Git + GitHub	Version control, one branch per member, pull requests for merging
Swagger UI / Postman	Testing API routes
Supabase dashboard	Viewing tables and running SQL changes
Figma	Wireframes and UI design

10. Running the System

Backend

```
cd backend
npm install
npm run dev        # server on http://localhost:5000
```

Mobile app

```
flutter pub get
flutter run        # on an Android emulator or phone
```

The backend needs a .env file with the database address (DATABASE_URL) and a token secret (JWT_SECRET).